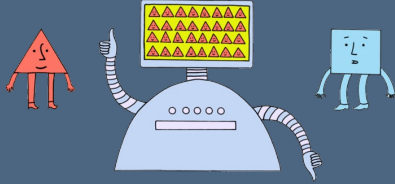


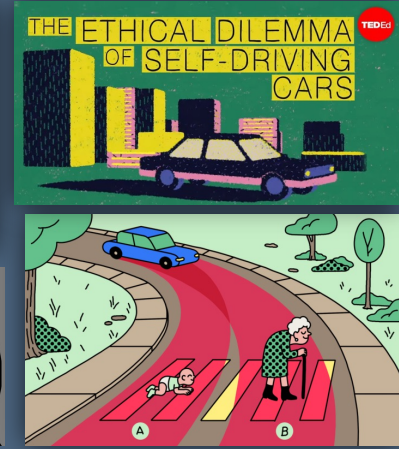
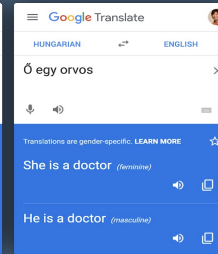
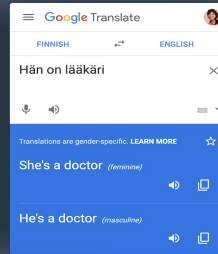
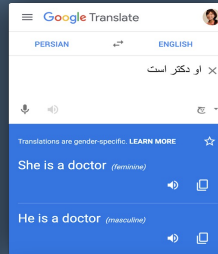
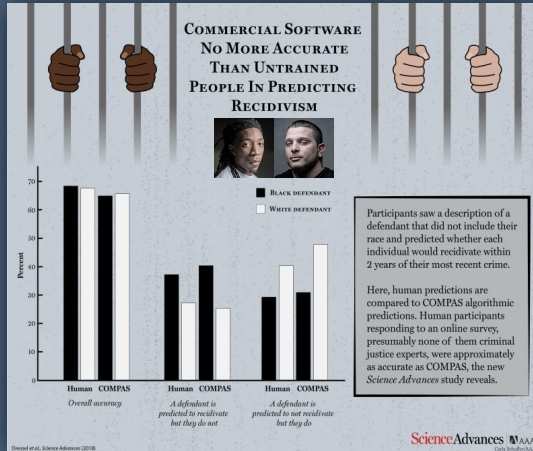
ReFAIR: Toward a Context-Aware Recommender for Fairness Requirements Engineering

Carmine Ferrara, Francesco Casillo, Carmine Gravino, Fabio Palomba, Andrea De Lucia

In today's software production, data-driven machine learning (ML) algorithms are more and more employed to support decision-making activities performed by individuals and companies.



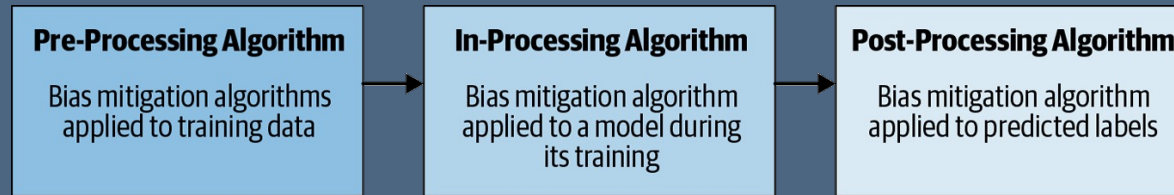
Despite the benefits brought by ML algorithms, researchers have been reporting on the implications that those algorithms may have on ethics and fairness: the reliance on historical data may lead a ML algorithm to gather biased knowledge about the relations ruling a phenomenon, which might lead to unfair predictions and recommendations that, in turn, might reiterate discrimination and injustice.



Software fairness refers to the quality or state of being just, unbiased, and equitable. It involves treating individuals or groups without discrimination or favoritism, and ensuring that decisions or actions are guided by principles of equality, impartiality, and objectivity.

Fairness can be applied in various contexts, such as in social, economic, or legal situations, and is often associated with concepts like justice, equality, and human rights. Overall, fairness is a fundamental principle of a just and harmonious society.

*the Software Engineering for Artificial Intelligence (**SE4AI**) research community has been actively working on this matter, arguing the need for novel engineering processes to treat **fairness** throughout the machine learning lifecycle.*



Pre-Processing Algorithm

Bias mitigation algorithms
applied to training data

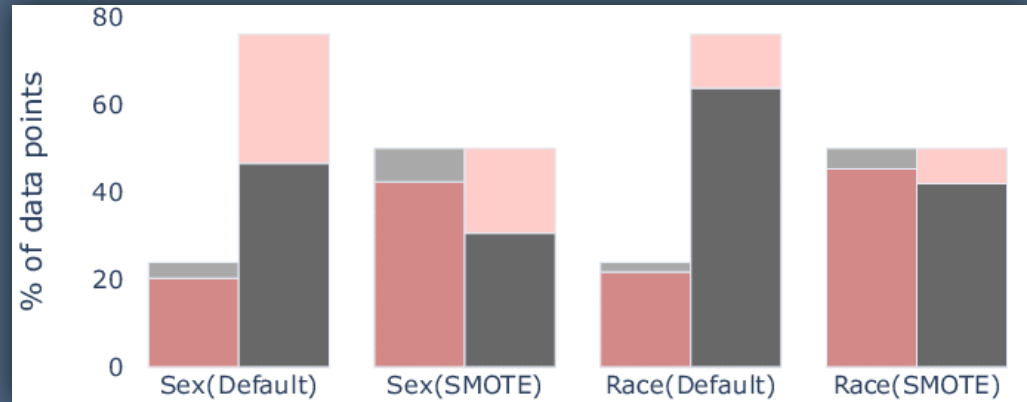
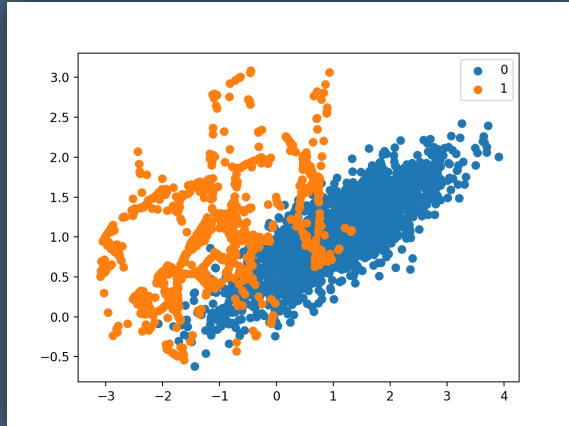
Chakraborty et al. proposed FAIR-SMOTE,
an oversampling algorithm able to
rebalance training data according to
sensitive attribute groups

joymallyac/Fair- SMOTE



GitHub repo for FSE 2021 Paper - "Bias in
Machine Learning Software: Why? How? What to
do?"

Rk 1 Contributor 0 Issues 9 Stars 7 Forks

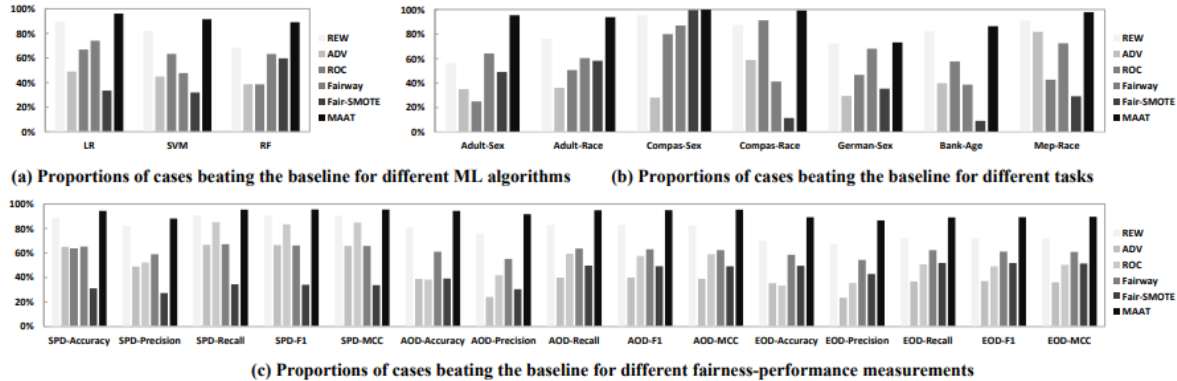
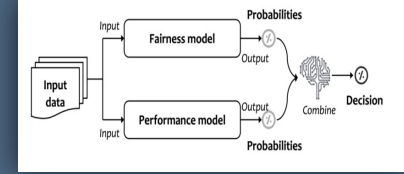


Chakraborty, Joymallya, Suvodeep Majumder, and Tim Menzies. "Bias in machine learning software: Why? how? what to do?." *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2021.

In-Processing Algorithm

Bias mitigation algorithm
applied to a model during
its training

Chen et al. propose MAAT, a novel ensemble approach to improving fairness-performance trade-off for ML software.

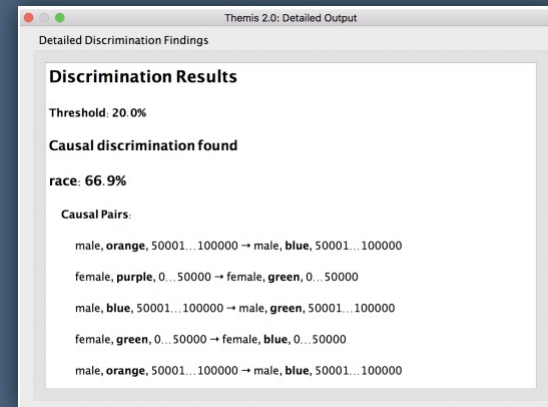
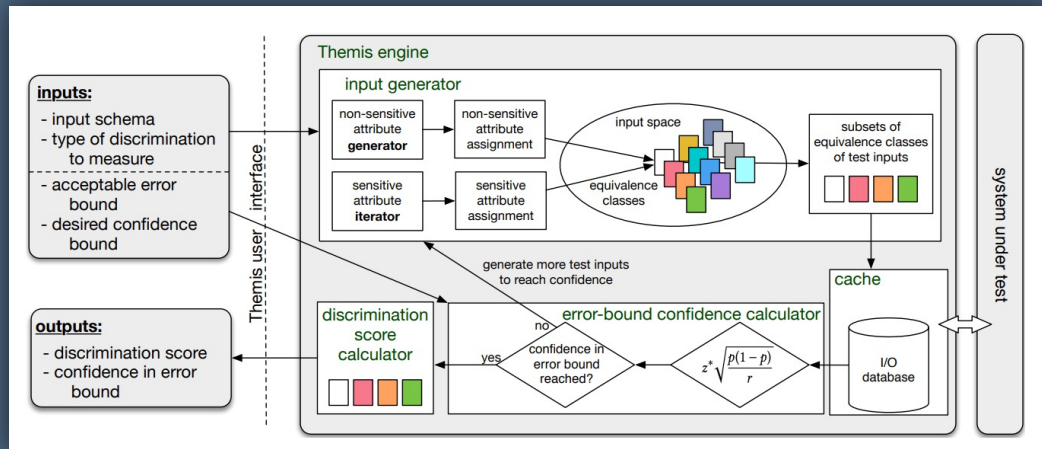


Zhenpeng Chen, Jie M. Zhang, Federica Sarro, and Mark Harman. 2022. MAAT: a novel ensemble approach to addressing fairness and performance bugs for machine learning software. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2022)*.

Post-Processing Algorithm

Bias mitigation algorithm applied to predicted labels

Angell et al. designed and developed Themis, an automated test suite generator to measure two types of discrimination, including causal relationships between sensitive inputs and program behavior.



LASER-UMASS/ Themis



Themis™ is a software fairness tester.

5 Contributors 10 Issues 96 Stars 19 Forks

Rico Angell, Brittany Johnson, Yuriy Brun, and Alexandra Meliou. 2018. Themis: automatically testing software for discrimination. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2018)*. Association for Computing Machinery, New York, NY, USA

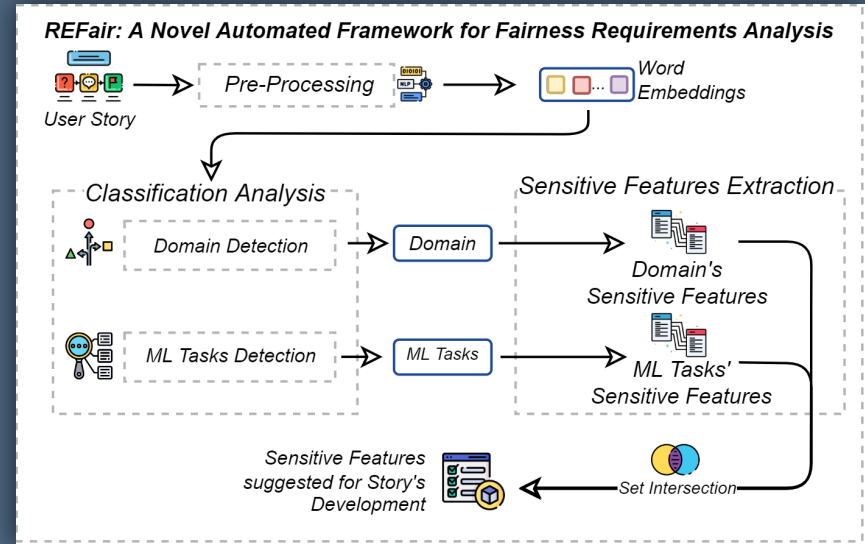
Nonetheless, most of the existing approaches can only be used in late development stages, e.g., during training or testing activities, while there is a lack of methods that may provide practitioners with early fairness analytics and, therefore, enable the treatment of fairness throughout the development lifecycle.

To perform the first step toward this objective, in this work we proposed REFAIR, a novel context-aware requirements engineering framework able to classify sensitive features from User Stories.

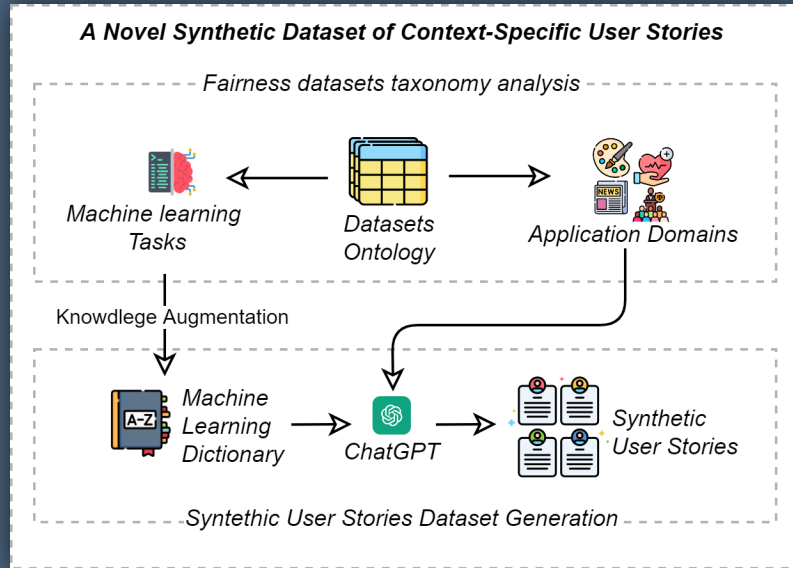
The first challenge our work consisted in the identification of a dataset of requirements pertaining to the construction of ML-enabled systems.

Such a dataset should have had specific properties:

- It should have reported domain-specific requirements of ML-enabled systems;
- It should have been diverse enough to investigate software fairness requirements engineering in various domains
- large enough to experiment with our approach, and generic enough not to be explicitly tailored on fairness analysis



Unfortunately, the current literature does not offer any off-the-shelf solution. Hence, we proceeded with the creation of a synthetic dataset of requirements.



We first selected a requirements format. Among the available, we focused on User Stories (USs), that represent a widely adopted instrument to describe software features from the perspective of the users that will interact with the system being developed.

The second step toward the definition of our synthetic dataset consisted of the mapping of the existing knowledge concerned with the application domains and ML tasks for which software fairness concerns arise.

We exploited resources two different published resources, i.e. a fair specific dataset taxonomy and a public machine learning dictionary as starting assets.

Fabris, A., Messina, S., Silvello, G., & Susto, G. A. (2022). Algorithmic fairness datasets: the story so far. *Data Mining and Knowledge Discovery*, 36(6), 2074-2152.

N. Duran-Silva, E. Fuster, F. A. Massucci, C. Parra-Rojas, Arnau Quinquill`a, F. Roda, B. Rondelli, N. Bovenzi, and C. Toietta, "A controlled vocabulary for research and innovation in the field of artificial intelligence (ai)," 2021

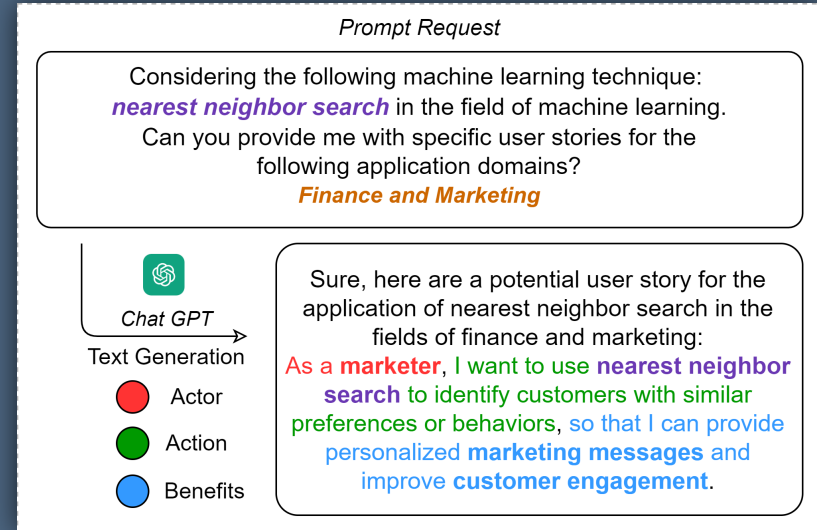
Obtained the starting application domains and machine learning techniques, we generated 12,401 user stories, providing to Chat GPT a well-defined textual prompt:

Considering the following:

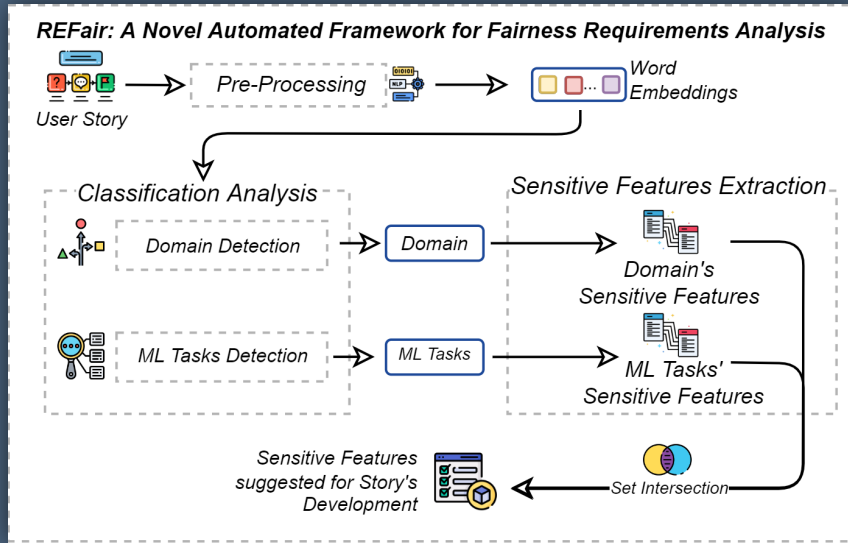
[High-level machine learning task]
OR [Low-level machine learning technique]

in the field of [machine learning]
OR [natural language processing].

Can you provide me with specific user stories
for the following application domains?
[List of Relevant Application Domains]



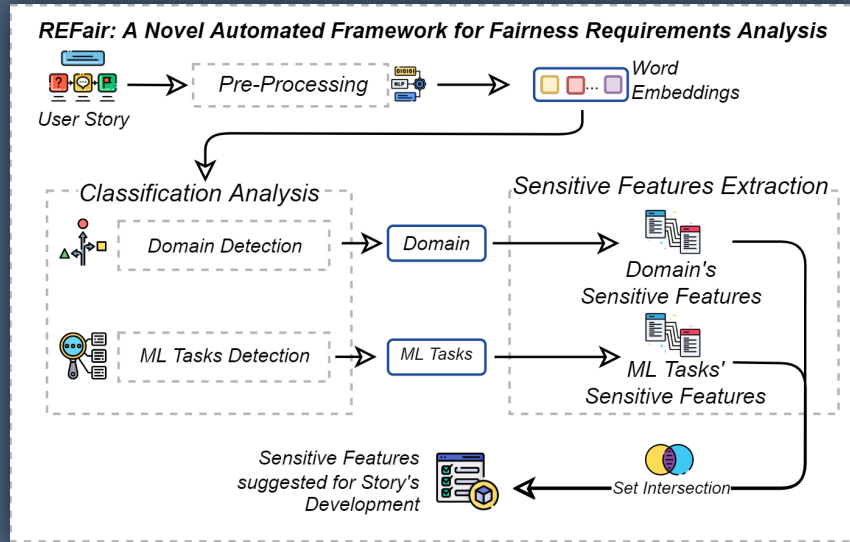
Sequentially, our focus moved to the identification of the building blocks to design and develop our *Requirements Engineering for FAIRness analytics approach*:



The first step of ReFAIR allows to produce a N-dimensional space representation of the input US: in turn, such a representation enables the extraction of features out of the text that natural language models can use for classification purposes.

Our framework was tested on different multiple word embedding techniques such as TF-IDF, BERT, WORD2VEC, FASTTEXT, and GLOVE.

Sequentially, our focus moved to the identification of the building blocks to design and develop our *Requirements Engineering for FAIRness analytics approach*:



The word embeddings are then taken as input by the classification analysis modules of ReFAIR.

This comprises two main components, namely those that operationalize the classification of application domains and ML techniques identifiable in a User Story.

To select specific sub-optimal algorithm for domain and task classification, we conducted a formal empirical evaluation considering two specific Research Questions.

Q RQ₁. *To what extent can REFAIR classify ML-specific application domains from user stories?*

Q RQ₂. *To what extent can REFAIR classify ML-specific tasks from user stories?*

Domains Classification

Q RQ₁. *To what extent can REFAIR classify ML-specific application domains from user stories?*

We defined the application domain identification task as a supervised *multi-class* learning problem.

Our experiment consisted in evaluating 125 ML algorithms and word embedding combinations to address RQ1.

We applied a formal ten-fold cross validation to split our synthetic dataset in ten folds and evaluate the performance of the performance of each of the 125 learners on each fold, computing the F1-Score and the Accuracy of each learning model.

Comparison of selected embedding techniques for the Domain Detection classifier.														
TF-IDF			BERT			Word2Vec			FastText			Glove		
Model	F1-Score	Accuracy	Model	F1-Score	Accuracy	Model	F1-Score	Accuracy	Model	F1-Score	Accuracy	Model	F1-Score	Accuracy
ET	0.80	0.80	XGBC	0.98	0.98	CCCV	0.91	0.91	SVC	0.94	0.94	CCCV	0.91	0.91
SVC	0.80	0.80	BC	0.98	0.98	LR	0.91	0.91	CCCV	0.94	0.94	LR	0.91	0.91
CCCV	0.80	0.80	DT	0.98	0.98	LSVC	0.90	0.90	LR	0.94	0.94	LDA	0.90	0.90

To the best of our results, we selected the combination of BERT and XGB CLASSIFIER, which exhibited Accuracy and F1-Score close to 98% in the domain detection task.

ML- Classification Task classification

Q RQ₂. *To what extent can REFAIR classify ML-specific tasks from user stories?*

We defined the application domain identification task as a supervised *multi-label* learning problem.

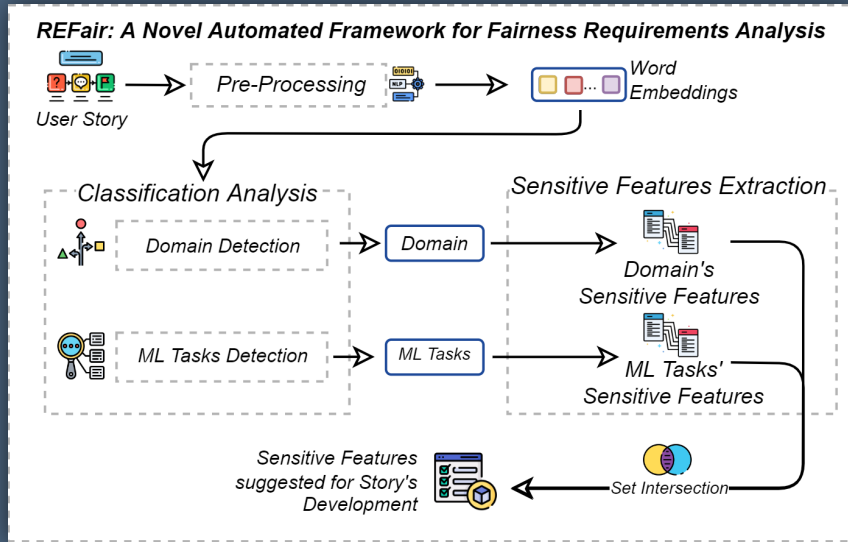
Our experiment consisted in evaluating 90 ML algorithms and word embedding combinations to address RQ2.

We applied a formal ten-fold cross validation to split our synthetic dataset in ten folds and evaluate the performance of the performance of each of the 90 learners on each fold, computing the F1-Score and the Hamming-Loss of each learning model.

Comparison of selected embedding techniques for ML Task Detection classifier.														
TF-IDF			BERT			Word2Vec			FastText			Glove		
Technique + Model	F1-Score	Hamming Loss	Technique + Model	F1-Score	Hamming Loss	Technique + Model	F1-Score	Hamming Loss	Technique + Model	F1-Score	Hamming Loss	Technique + Model	F1-Score	Hamming Loss
LP + RF	0.88	0.06	LP + DT	0.86	0.07	LP + LSVC	0.86	0.07	LP + LSVC	0.81	0.09	LP + LSVC	0.90	0.05
LP + DT	0.82	0.09	LP + RF	0.84	0.08	LP + GNB	0.76	0.12	LP + GNB	0.81	0.10	LP + GNB	0.72	0.15
BR + RF	0.80	0.09	BR + DT	0.78	0.11	BR + KNN	0.68	0.15	BR + KNN	0.74	0.12	BR + KNN	0.67	0.15

To the best of our results, the combination of LABEL POWERSET, GLOVE, and LINEAR SUPPORT VECTOR CLASSIFICATION established an F1-Score slightly above 90%, with a Hamming Loss of 5% on the ML tasks classification.

Sequentially, our focus moved to the identification of the building blocks to design and develop our *Requirements Engineering for FAIRness analytics approach*:

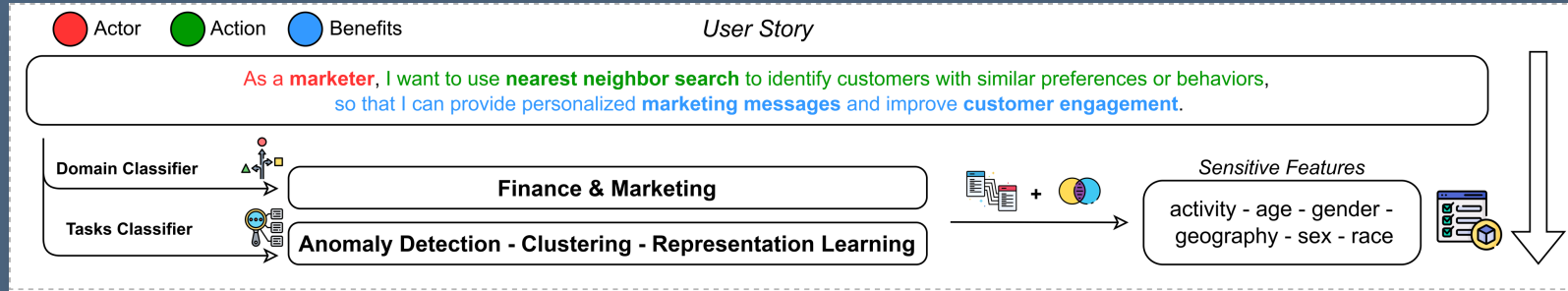


The application domain and ML tasks classified in the previous step are finally used to recommend sensitive features.

REFAIR exploits specific mapping we defined from the literature to identify the sensitive features connected to both the application domain and ML tasks concerned with the the classified domain.

The final outcome of our Framework comprises the set of sensitive features relevant when jointly considering the application domain and the learning tasks related to it.

A practical example of the ReFAIR approach's execution



The figure reports a running example of our framework, which shows how it could recommend sensitive features for an input.

The approach correctly classifies **Finance & Marketing** as the story domain and identifies **Anomaly Detection, Clustering, and Representation Learning** as possible ML tasks through which the story may be implemented.

Finally, for each ML task, ReFair maps relevant sensitive features, finally recommending working activity, age, etc. as the set of sensitive features to consider in the subsequent implementation steps of the US.